Course Name		
Computer Programming Lab		
Lab Title		
Reading and Writing CSV files with and without Pandas library / Python Libraries		
Name	**Registration Number**	**Lab #**
M.Hamzah Iqbal	F24604018	11/12
Date	**Instructor's Name**	**Instructor's Signature**
	LE Ayesha Ashfaque	

Lab #11: Reading and Writing CSV files with and without Pandas library

---

Objective:

- To have basic understanding of sorting and searching arrays in Python.
- To Implement bubble sort in Python.

Tools/Software Requirement:

Python 3 Anaconda

Introduction:

A CSV file (Comma Separated Values file) is a type of plain text file that uses specific structuring to arrange tabular data. The structure of a CSV file is given away by its name. Normally, CSV files use a comma to separate each specific data value. Here's what that structure looks like:

```
CSV
```

```
column 1 name,column 2 name, column 3 name
first row data 1,first row data 2,first row data 3
second row data 1,second row data 2,second row data 3
...
```

CSV files are normally created by programs that handle large amounts of data. They are a convenient way to export data from spreadsheets and databases as well as import or use it in other programs. For example, you might export the results of a data mining program to a CSV file and then import that into a spreadsheet to analyze the data, generate graphs for a presentation, or prepare a report for publication.

CSV files are very easy to work with programmatically. Any language that supports text file input and string manipulation (like Python) can work with CSV files directly.

Parsing CSV Files With Python's Built-in CSV Library

The csv library provides functionality to both read from and write to CSV files. Designed to work out of the box with Excel-generated CSV files, it is easily adapted to work with a variety of CSV formats. The csv library contains objects and other code to read, write, and process data from and to CSV files.

Reading CSV Files With csv:

Reading from a CSV file is done using the reader object. The CSV file is opened as a text file with Python's built-in open() function, which returns a file object. This is then passed to the reader, which does the heavy lifting.

Here's the employee_birthday.txt file:

CSV

```
name,department,birthday month
John Smith,Accounting,November
Erica Meyers,IT,March
```

Here's code

Python

```
import csv

with open('employee_birthday.txt') as csv_file:
    csv_reader = csv.reader(csv_file, delimiter=',')
    line_count = 0
    for row in csv_reader:
        if line_count == 0:
            print(f'Column names are {", ".join(row)}')
            line_count += 1
        else:
            print(f'\t{row[0]} works in the {row[1]} department, and was born in {row[2]}.')
            line_count += 1
    print(f'Processed {line_count} lines.')
```

This results in the same output as before:

Shell

```
Column names are name, department, birthday month
    John Smith works in the Accounting department, and was born in November.
    Erica Meyers works in the IT department, and was born in March.
Processed 3 lines.
```

Writing CSV Files With csv:
You can also write to a CSV file using a writer object and the .write_row() method:

```python
import csv

with open('employee_file.csv', mode='w') as employee_file:
    employee_writer = csv.writer(employee_file, delimiter=',', quotechar='"', quoting=csv.QUOTE_MINIMAL)

    employee_writer.writerow(['John Smith', 'Accounting', 'November'])
    employee_writer.writerow(['Erica Meyers', 'IT', 'March'])
```

Lab #12: Python Libraries

.

Objective:

- To have a basic understanding of Multiple Libraries
- To Implement the Libraries in Anaconda

Tools/Software Requirement:

Python 3 Anaconda

Introduction:

Python libraries are collections of pre-written code modules that provide additional functionalities for Python programs. They contain ready-to-use functions, classes, and methods that can be imported and used in your code. Python libraries are designed to address specific tasks or domains, such as data analysis, web development, machine learning, and more. They enhance your productivity by saving you time and effort in writing code from scratch. Popular Python libraries include NumPy for numerical computing, Pandas for data manipulation, Matplotlib for data visualization, Flask for web development, and TensorFlow for machine learning. These libraries expand the capabilities of Python and allow you to leverage existing code and expertise in specific domains, making your programming journey more efficient and powerful.

The popular Python libraries:

1. NumPy: Numerical computing library for arrays and mathematical operations.
2. Pandas: Data manipulation and analysis library with powerful data structures.
3. Matplotlib: Data visualization library for creating charts, plots, and graphs.
4. Scikit-learn: Machine learning library for classification, regression, and clustering.
5. Requests: Library for making HTTP requests and handling API interactions.
6. Pillow: Image processing library for opening, manipulating, and saving images.
7. OpenCV: Computer vision library for image and video analysis and manipulation.

These libraries cover a range of domains, from data analysis and machine learning to web development, image processing, and more. Each library provides specialized tools and functionalities to simplify and enhance your Python programming experience in their respective domains.

Scikit-Learn:

Scikit-learn, also known as sklearn, is a popular Python library for machine learning. It provides a wide range of tools and algorithms for data mining, data analysis, and machine learning tasks. Scikit-learn is built on top of other scientific libraries in Python, such as NumPy and SciPy, and it integrates well with the broader Python ecosystem.

Scikit-learn offers a comprehensive set of functionalities for various machine learning tasks, including:

1. **Classification:** Scikit-learn provides algorithms for classifying data into different predefined categories. It includes popular classifiers such as Support Vector Machines (SVM), Random Forests, and Naive Bayes.

2. **Regression:** Scikit-learn supports various regression algorithms to predict continuous values based on input features. Linear regression, Decision Trees, and Gradient Boosting are among the regression models available.

3. **Clustering:** Scikit-learn includes clustering algorithms for grouping data points into clusters based on similarity. K-Means, DBSCAN, and hierarchical clustering are some of the clustering methods offered.

4. **Dimensionality Reduction**: Scikit-learn offers techniques for reducing the number of features in a dataset while preserving important information. Principal Component Analysis (PCA) and t-SNE are commonly used for dimensionality reduction.

5. **Model Selection and Evaluation:** Scikit-learn provides tools for model selection and evaluation, including methods for cross-validation, performance metrics, and hyperparameter tuning.

6. **Preprocessing**: Scikit-learn offers a range of data preprocessing techniques, such as feature scaling, handling missing values, and encoding categorical variables.

Request Library:

1. **HTTP Methods**: The requests library supports various HTTP methods, including GET, POST, PUT, DELETE, and more. It allows you to send requests to web servers and interact with resources.

2. **Request Parameters**: You can pass parameters with your requests, such as query parameters, request headers, cookies, and authentication credentials. The requests library provides a straightforward way to set and manage these parameters.

3. **Response Handling:** After sending a request, the requests library handles the HTTP response and provides convenient methods and attributes to access response data. You can retrieve response content, headers, status codes, and more.

4. **Session Management**: The requests library supports sessions, allowing you to persist certain parameters across multiple requests, such as cookies or session headers. This is useful when working with APIs that require authentication or maintain session state.

5. **Error Handling:** The requests library includes mechanisms to handle various types of errors, such as connection errors, timeouts, and HTTP errors (e.g., 404 or 500). You can catch and handle these errors gracefully in your code.